Vercel Deployment Notes

This repository can be deployed to Vercel as a single FastAPI application by using the root `index.py` entrypoint.

What this deployment supports

- FastAPI API routes
- HTML pages served by the backend from `frontend/**`
- Dashboard, history, credentials, and status pages
- External PostgreSQL-backed data access if `DATABASE_URL` is configured

What does not fit Vercel well

This project was originally built for Docker-based deployment. The following parts are not a good match for Vercel Functions and should be moved to a worker platform or VM if you need full functionality:

- Vercel deploys the FastAPI app as a single Python function bundle, so bundle size and function duration limits apply
- Long-running multi-cloud scans
- The scheduler worker in `backend/scheduler_worker.py`
- Docker-installed scanner binaries such as Trivy, Grype, Nuclei, OWASP ZAP, Gitleaks, and CloudFox
- Bundled PostgreSQL and Redis services from `docker-compose.yml`

Required external services

- Managed PostgreSQL for `DATABASE_URL`
- Optional managed Redis if you later split out background jobs
- Optional object storage / secrets manager for large artifacts and credentials

Suggested Vercel environment variables

Required for basic boot:

- `DATABASE_URL`
- `SECRET_KEY`
- `ENCRYPTION_KEY`
- `SSO_LOGIN_URL`
- `SSO_SCANNER_REDIRECT_URL`
- `SSO_VERIFY_URL`
- `SCAN_WORKER_URL`
- `SCAN_WORKER_TOKEN`

Strongly recommended:

- APP_ENV=production
- LOG_LEVEL=INFO
- ALLOWED_HOSTS
- CORS_ORIGINS

Optional feature flags / integrations:

- OPENAI_API_KEY
- OPENAI_AGENT_MODEL
- AWS_REGION
- AWS_PROFILE
- GCP_PROJECT_ID
- GCP_SERVICE_ACCOUNT_JSON

Deploy steps

1. Create a Vercel project from this repository.
2. Add the environment variables above in the Vercel dashboard.
3. Point DATABASE_URL at a managed PostgreSQL instance.
4. Deploy with `vc deploy` or through the Git integration.
5. Test /health, /api/info, /dashboard, and /api/system/status.

SSO companion app

CloudGuard now expects the companion DeepTrustxAI SSO portal from:

- sso/new/deeptrustxai

Recommended production values:

- SSO_LOGIN_URL=https://cloudguard-sso.vercel.app/
- SSO_SCANNER_REDIRECT_URL=https://cloudguard-sso.vercel.app/api/scan/redirect
- SSO_VERIFY_URL=https://cloudguard-sso.vercel.app/api/sso/verify

The legacy folder:

- sso/sso-login

should be treated as an older implementation reference, not the active public SSO frontend.

Recommended production shape

Best fit:

- Vercel: frontend + FastAPI control plane
- Managed Postgres: app data
- Separate worker host (Render, Fly.io, Railway, ECS, or a VM): actual cloud scanning, CloudFox, Trivy, scheduler, and other long-running jobs

Worker deployment details:

- `WORKER_DEPLOY.md`

This repository now includes that split scaffold: the Vercel app can forward heavy scan execution to the dedicated worker when `SCAN_WORKER_URL` is set.